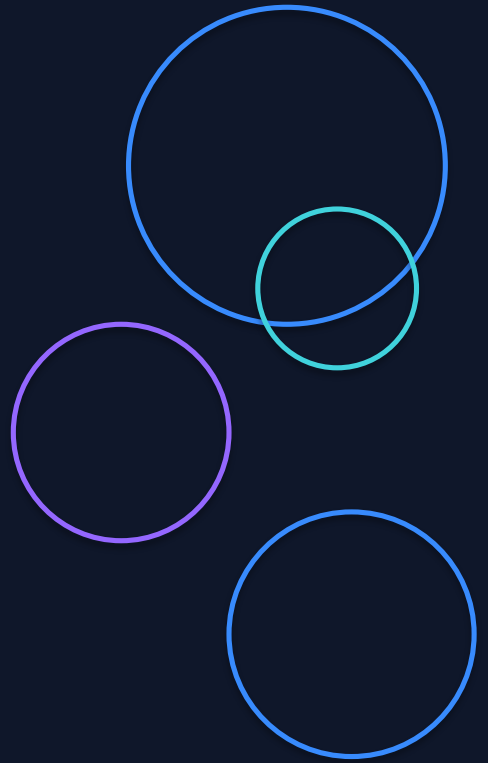
Image Interpolation Using Neural Networks

A Comparative Study of Classical Methods, SRCNN and our neural network



Why interpolation is not enough

Upscaling increases image size—but not necessarily the detail we want.

Classical interpolation

Fast, deterministic pixel estimation. It cannot recover details lost during downscaling.

Neural super-resolution

Learns an LR → HR mapping from examples and can estimate plausible missing detail.

The challenge

Sharper kernels can also amplify noise; quality must be measured consistently.

Research question

How much can a compact CNN improve image quality over standard resizing methods—and which network components are responsible?

Objectives & experimental scope

01 Reproducible pipeline

Downscale HR images, optionally add Gaussian noise, upscale, and score against the original.

02 Fair comparison

Compare four classical methods, SRCNN and our_nn at ×2 and ×4.

03 Ablation study

Switch components off to estimate their contribution to PSNR.

DATASETS

DIV2K

800 training images

Set5

5 test images

Set14

14 images

BSD100

100 images

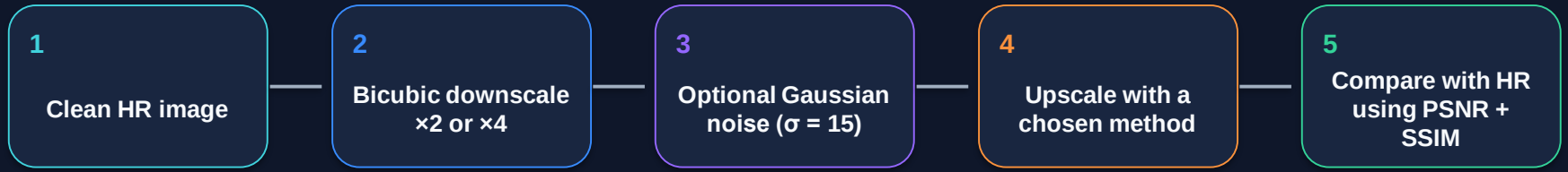
Urban100

100 urban scenes

Evaluation uses luminance-channel PSNR (dB) and SSIM (0–1).

Experimental pipeline

All methods receive the same degraded input and are compared against the untouched HR image.



Reproducibility

Fixed random seed for noise; image crops divisible by scale factor.

Implementation

PyTorch models and experiment settings are registered by name and YAML configuration.

Methods compared

From simple pixel copying to learned detail reconstruction.

Nearest-neighbour

Copies the closest pixel.

Bilinear

Weighted average of the nearest 2 × 2 pixels.

Bicubic

Cubic interpolation over a 4 × 4 neighbourhood.

Lanczos

Windowed sinc filter; often sharper, but may amplify noise.

SRCNN

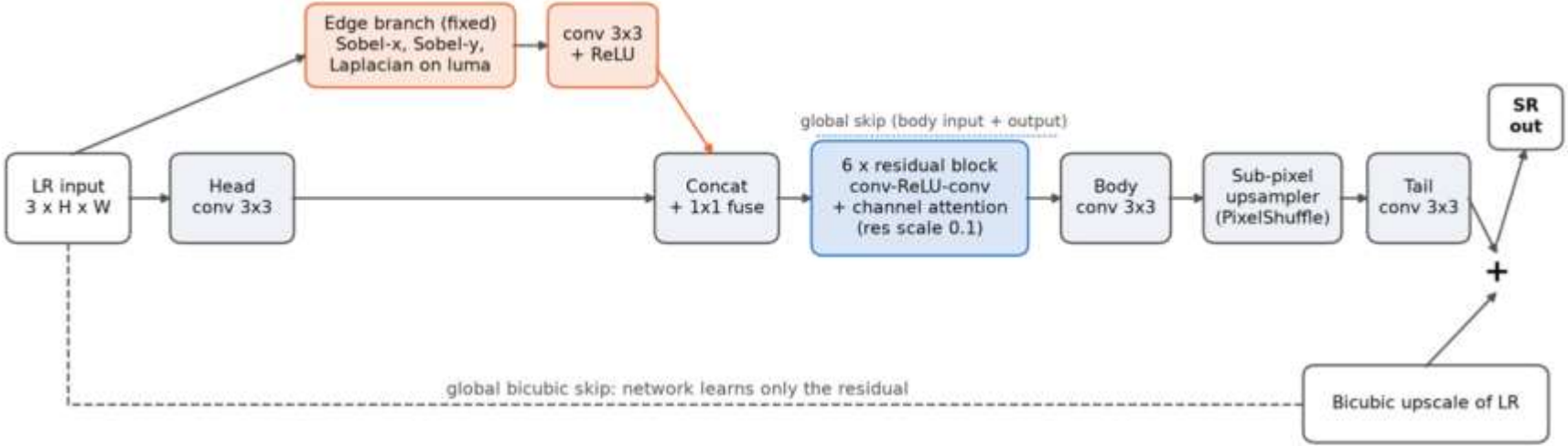
Three convolution layers applied after bicubic upscaling; 20,099 parameters.

our_nn

Lightweight residual CNN with bicubic skip and edge prior; ~200K parameters.

Classical methods require no training while neural methods learn from HR training patches.

our_nn architecture (n_feats = 32)



Training & evaluation

Training data

DIV2K • random 96 × 96 HR patches
Flips + 90° rotations
8,000 patches per epoch

Optimisation

L1 loss + Adam
100 epochs • batch size 16 for our_nn
Learning rate halved every 40 epochs

Validation

Set14 used for checkpoint selection
Best validation PSNR checkpoint retained
Seed 42 • Kaggle T4 GPU

PSNR — pixel-level reconstruction

$\text{PSNR} = 10 \log_{10}(255^2 / \text{MSE})$
Higher dB means lower pixel error.

SSIM — structural similarity

Scores local luminance, contrast and structure.
Higher is better; range 0–1.

Scores are computed on the luminance (Y) channel and averaged equally across all test sets.

our_nn leads across all tested settings

Mean scores across Set5, BSD100 and Urban100.



×2 clean

32.74 dB • +2.40 dB vs bicubic

×4 clean

27.36 dB • +1.32 dB vs bicubic

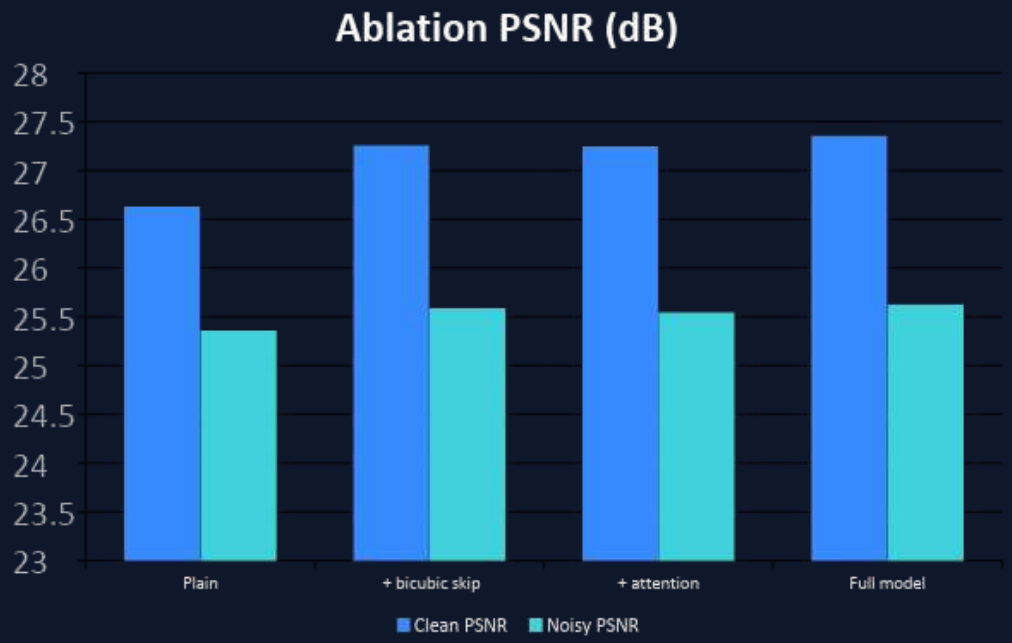
×4 noisy (σ = 15)

25.63 dB • +1.11 dB vs bicubic

×4 clean SSIM: 0.764 vs 0.714 for bicubic

Which components actually help?

×4 mean PSNR; each configuration was trained once.



Bicubic skip

Largest gain: +0.62 dB clean; +0.23 dB noisy.

Channel attention

Negligible improvement.

Edge branch

Small gain: +0.11 dB clean and +0.09 dB noisy.

Quality, speed & limitations

Urban100 (×4)

SRCNN: 31.8 ms/image
our_nn: 24.9 ms/image

Set5 (×4)

SRCNN: 7.3 ms/image
our_nn: 11.0 ms/image

Why timing varies

our_nn works at LR resolution and upscales at the end; runtime depends on image size.

Urban100 / img_001_SRF_4_HR, x4 clean (crop 80x80 px of the HR image)



Key takeaways

Neural methods win

our_nn outperforms classical interpolation and the project's SRCNN in every tested setting.

Skip connection matters

Adding a bicubic baseline and learning a residual gives the largest measured gain.

Noise remains difficult

Noise lowers scores for all methods; models trained on noisy data suppress some of it.

Thank you • Questions?